

UNIT 1 – JAVA (DETAILED NOTES)

◇ 1. Introduction to Java

✓ Why Java?

Java is popular because:

- **Platform Independent** → *Write Once, Run Anywhere (WORA)*
- **Object-Oriented** → Better code structure
- **Secure** → No direct memory access (no pointers like C++)
- **Robust** → Strong memory management + exception handling
- **Multithreading** → Can run multiple tasks simultaneously

💡 Real-life example:

Think Java like **YouTube app** → runs on Android, Windows, Mac
Same code → different systems

✓ History of Java

- Developed by James Gosling at Sun Microsystems (1995)
- Originally called **Oak**
- Later owned by Oracle Corporation

✓ JVM, JRE, JDK (VERY IMPORTANT)

◇ JVM (Java Virtual Machine)

- Runs Java bytecode
- Converts bytecode → machine code

👉 Like a **translator between Java and OS**

◇ JRE (Java Runtime Environment)

- JVM + Libraries
- Used to **run Java programs**

◇ JDK (Java Development Kit)

- JRE + Compiler + Tools
- Used to **develop Java programs**

Flow:

.java → (Compiler) → .class → (JVM) → Output

✓ Java Environment

- Set **PATH variable**
- Install JDK
- Use commands:

```
javac file.java  
java file
```

✓ Java Source File Structure

```
package mypack;  
  
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Structure:

1. Package
2. Import
3. Class
4. main()

◇ 2. Programming Structures in Java

✓ Class Definition

```
class Student {  
    int id;  
    String name;  
}
```

👉 Class = Blueprint (like structure in C)

✓ Constructors

```
class Student {  
    Student() {  
        System.out.println("Constructor called");  
    }  
}
```

👉 Runs automatically when object is created

✓ Methods

```
void display() {  
    System.out.println("Hello");  
}
```

✓ Access Specifiers

Specifier	Access
public	Everywhere
private	Same class
protected	Same package + subclass
default	Same package

✓ Static Members

```
class Test {  
    static int count = 0;  
}
```

👉 Shared by all objects

✓ Final Members

```
final int x = 10;
```

👉 Cannot be changed

✓ Comments

```
// Single line  
/* Multi-line */
```

✓ Data Types

Primitive:

- int, float, double, char, boolean

Non-Primitive:

- String, Array, Class

✓ Variables

```
int a = 10;
```

Types:

- Local
- Instance
- Static

✓ Operators

Type	Example
Arithmetic	+, -, *
Relational	>, <
Logical	&&,
Assignment	=

✓ Control Flow

```
if(a > 10) { }  
  
for(int i=0;i<5;i++) { }  
  
while(true) { }
```

✓ Arrays

```
int arr[] = {1,2,3};
```

✓ Strings

```
String s = "Hello";
```

👉 Immutable (cannot change)

◇ 3. Object-Oriented Programming (VERY IMPORTANT)

✓ Class & Object

```
Student s = new Student();
```

👉 Object = instance of class

✓ Inheritance

```
class A { }  
class B extends A { }
```

👉 Reuse code

✓ Super Class & Sub Class

- Parent → Superclass
- Child → Subclass

✓ Method Overriding

```
class A {  
    void show() { }  
}  
class B extends A {  
    void show() { }  
}
```

👉 Same method, different behavior

✓ Method Overloading

```
void add(int a, int b) { }  
void add(int a, int b, int c) { }
```

👉 Same name, different parameters

✓ Encapsulation

```
class Student {  
    private int id;
```

```
    public void setId(int id) {  
        this.id = id;  
    }  
}
```

👉 Data hiding

✓ Polymorphism

👉 One name, many forms

Example:

```
add(int a, int b)  
add(double a, double b)
```

✓ Abstraction

👉 Hide implementation, show only functionality

✓ Interface

```
interface A {  
    void show();  
}
```

👉 Only method declaration

✓ Abstract Class

```
abstract class A {  
    abstract void show();  
}
```

👉 Can have abstract + normal methods

◇ 4. Packages (IMPORTANT FOR EXAM)

✓ Defining Package

```
package mypack;
```

✓ CLASSPATH

👉 Tells JVM where to find classes

Example:
`set classpath=.`;

✓ Creating JAR Files

```
jar cvf mylib.jar *.class
```

👉 Used to distribute libraries

✓ Import

```
import java.util.*;
```

✓ Static Import

```
import static java.lang.Math.*;
```

👉 Direct use:
`sqrt(16);`

✓ Naming Convention

- Reverse domain style:
`com.company.project`